

Week 5 – Tell the Story of Objects (Part 2)

Business Case: Online Event Ticketing System (Spot Saver)

Group Members: Aeen, Stan

Introduction

In Week 3 we first learned about domain models and built a basic version of our system. During Week 4 we refined that diagram by focusing on clearer class names, key attributes, and logical relationships. Now, in Week 5, we continued that process by applying the new theory about conceptual class diagrams. We improved the realism of our model while still keeping it conceptual and readable for a first-year level. The goal was to “tell the story” of how objects in our Online Event Ticketing System relate to each other in a real situation.

Updated Conceptual Model

The refined model still centers on seven main classes — **Customer**, **Order**, **OrderLine**, **Ticket**, **TicketType**, **Event**, **Venue** — and also includes **Payment** and **Organizer** as supporting concepts.

Each class has meaningful attributes, for example:

- *Customer*: customerId, name, email, phoneNumber
- *Order*: orderId, orderDate, totalAmount, status (OrderStatus)
- *Ticket*: ticketId, qrCode, status (TicketStatus), seatNumber?
- *Event*: eventId, title, date, category (EventCategory)
- *Venue*: venueId, name, location, capacity

The **associative class OrderLine** connects *Order* and *TicketType*. It represents each line item inside an order, with attributes such as *quantity*, *unitPrice*, and *lineTotal*. This small addition fixes the common problem of representing multiple ticket types within a single order.

We also introduced **enumeration classes** to make data values consistent:

- *OrderStatus* → Pending | Paid | Cancelled
- *TicketStatus* → Reserved | Issued | Cancelled
- *PaymentMethod* → Card | Bank Transfer
- *EventCategory* → Concert | Sport | Theatre

These enumerations limit invalid entries and clarify the state of each object without needing free-text fields.

Each relationship in the diagram uses clear multiplicities (for example, *one Customer can place many Orders, each Order has one Payment*).

Business Rules and Constraints

To make the model realistic, we added a few simple but useful rules as yellow “notes” in the diagram:

1. **BR1:** Tickets sold for an Event must be less than or equal to the Venue capacity.
2. **BR2:** An Order only becomes “Paid” when a Payment method is selected and a payment date exists.
3. **BR3:** Order.totalAmount = sum of all OrderLine.lineTotal values.

These rules make it clear that the diagram isn’t just a drawing — it also describes logical business behaviour.

Reflection and Conclusion

Compared to Week 4, the new version feels much closer to a working concept for an actual ticketing platform. The model now shows how every ticket purchase moves from a Customer to an Order, through Payment, and finally to an Event at a Venue created by an Organizer. The addition of enumerations and an associative class makes the structure cleaner and prevents data errors. This conceptual class diagram forms a strong foundation for the next phases of the SDLC, where we will later convert it into a design-level UML

diagram or database schema. For now, it clearly communicates **what** the system contains and **how** the objects relate — the core goal of conceptual modeling.

